Advanced Hibernate
Inheritance
Associations between entities.

Enter Advanced Hibernate (Project objective)

Inheritance in JPA/Hibernate
It supports 4 inheritance strategies .

First inheritance strategy :
1.Annotation : javax.persistence.MappedSuperclass
Class level annotation , to be added on abstract or concrete super class
Hibernate will NOT generate any table for it.
One can add all common fields in this class
All other entities can extend n inherit from the common super class

Which are typically required common members ?
ID related
creation date / time / time stamp
update date / time / time stamp
Refer - @CreationTimestamp , @UpdateTimeStamp
Version related (later can be added for optimistic locking)

2. Associations between Entities (HAS-A) : weaker form of association => aggregation (since Entities have a standalone life cycle , have a separate DB Identity in form of a seaprate table n it's PK)

Ref : Blogs E-R diagram

2.1 Establish bi dir , one->many n many->one association between the entities.
Category 1<----->* BlogPost
eg : BlogPost n Category

BlogPost : child , many, owning (since it contains FK column)
Category : parent , one , non-owning(inverse)

Technical terms : parent/child , one side/many , (non-owning)inverse side/owning

owning side of the association -- side which contains the FK (physical mapping)
non owning(inverse) side of the asso -- side which DOES NOT conatain the FK

2 Types of associations
uni directional
OR
bi directional

Directionality concept exists ONLY in Object world

Uni directional association : the one in which you can navigate only from 1 side to another.
Another eg : Course 1----->* Student

Bi directional association : you can navigate the association from any side.

eg : Course 1<----->* Student

How To configure a Bi dir relationship?

eg : Category 1 <------> * BlogPost
(Bi dir relationship , one to many n many to one)

Category : one , parent , non owning

BlogPost : many , child , owning (contains FK category_id ---> PK of depts table)

Steps
Configure Entities
1. Category : extends BaseEntity
Fields : name , description
+private List<BlogPost> empList=new ArrayList<>();
As per Gavin King's suggestion always init collection based property to empty collection (to avoid NullPointerException. while adding /removing the child elements)
generate : setters n getters.

2. BlogPost : extends BaseEntity
Fields : title,description,content
 +
private Category chosenCategory;
Add setter n getter

Problems n solutions

1.What will happen if you don't add any association mapping annotations?
Observation : org.hibernate.MappingException is thrown !
Why ? Hibernate CAN NOT figure out the type of association , between the entities.
Solution : Add mapping annotation .

JPA Annotations for E-R
@OneToOne
@ManyToOne
@OneToMany
@ManyToMany

2. After adding @ManyToOne n @OneToMany , no MappingException.
BUT how many tables were created by hibernate ? 3 (eg : categories,posts,categories_posts)

Is the link table really required for establishing one to many bi dir asso ? NO
Simpler way : FK

3. Reason behind additional table : Hibernate can't identify which is owning n inverse side of the association

Solution : In a bi dir association : It's mandatory to add mappedBy : property

in @OneToMany .

Which side should it appear : inverse(eg : Category)
What should be the value of mappedBy = Name of the association property , as it appears in the owning side
eg : In Category class : add
@OneToMany(mappedBy = "chosenCategory")
private List<BlogPost> empList = new ArrayList<>();

4. How to customize name n not null constraint of FK column ?
eg :
@JoinColumn(name="category_id",nullable=false)//optional BUT recommended
private Category chosenCategory;

5. Project Tip (suggestion from Gavin King)
In case of bi-dir associations , instead of adding complex logic in DAO or Tester ,
Add helper methods in the POJO layer itself
1. To add child entity
2. To remove a child entity
eg : In Category class ,
addBlog n removeBlog

Objectives
1. Add a new Category
i/p : name , description
o/p : mesg

2. Add a blog under specified category
i/p : blog details(title , desc , contents) +category id
o/p : mesg
Steps in Blog post DAO
Method declaration - String addNewBlog(Long categoryId, BlogPost post);

2.1 get category from it's id - get
2.2 valid category -
c1.addBlogPost(post);
session.persist(post);

Will you have to explicitly call : session.persist(blog) ?

Any simpler solution ? YES
Use cascade option

Cascading refers to the ability to automatically propagate the state of an entity across associations between entities.

eg , In current scenario , if Category is deleted , since it has a one-to-many relationship with BlogPost , you can define cascading to specify that when a Category is deleted, all of it's blog posts should be deleted as well. Or saved or updated.

Cascading in Hibernate refers to the automatic persistence of related entities.

When a change is made to an entity, such as an save /update or delete the changes can be cascaded to related entities as well.

Cascading can be configured using annotations

javax.persistence.CascadeType : enum
Values : ALL,PERSIST, MERGE, REMOVE, REFRESH, DETACH

Solution : Add a cascade type.
eg : @OneToMany (mappedBy = "chosenCategory",cascade = CascadeType.ALL)
private List<BlogPost> posts=new ArrayList<>();

2.6 Delete category details (lab work)
i/p : category name (unique)
o/p mesg
DB action - child recs should be deleted first(posts) n then the parent record(category)

(Test the effect of cascade on delete)
Hint - In CategoryDaoImpl
JPQL + getSingleResult
session.delete(category);
commit

Solve
Establish User(Blogger) 1<------* BlogPost
FK - author_id , not null
HOW ?

User -one , parent
BlogPost - many, child, owning

Any changes in BlogPost ?
@ManyToOne
@JoinColumn(name="author_id", nullable=false)
private User blogger;

Any changes in User ?
NONE

Add a blog under specified category , by specified blogger
i/p : blog details(title , desc , contents) , category id , blogger id
o/p : mesg
Steps in Blog post DAO
Method declaration - String addNewBlog(Long categoryId,Long bloggerId, BlogPost post);
steps - get category from it's id
get blogger from it's id
establish bi dir between Category -- BlogPost (category.addPost(post))
establish uni dir between User <-- BlogPost (post.setBlogger(user))
session.persist(post);- un necessary

3. Remove blog from the given category
i/p - blog id , category id
o/p - mesg
DB action - rec should be deleted from posts table

2.7 Remove blog post from a specific category
i/p : category id , post id
o/p mesg
How ?

Problem : When you try to remove a child from the parent(using removeBlogPost)
Hibernate will simply nullify the FK (firing update query) BUT will NOT delete the record .
Reason : Not yet specified orphanRemoval property to hibernate.

6. Set orphanRemoval=true on the Parent-Side

Setting orphanRemoval on the parent-side guarantees the removal of children without references.
It is good for cleaning up practice for dependent objects that should not exist without a reference from an owner object.

orphanRemoval : a property of @OneToMany
(Optional) Whether to apply the remove operation to entities that have been removed from the relationship and to cascade the remove operation to those entities.

6.5 Establish uni dir asso between
BlogPost n Blogger
BlogPost *----->1 User (Blogger)
How will you configure this in Entity layer ?

Steps -
1. In BlogPost class
add new field
@ManyToOne
@JoinColumn(name="author_id",nullable=false)
private User blogger;
2. getters n setters

Modified Objective
Add new blog post , along with the category n blogger

i/p : blog details...., category id , user id
o/p : mesg

7. Try to access Category details , using it's name
Print Category details n it's blog post details .

Problem : org.hibernate.LazyInitializationException

WHY ?

JPA/Hibernate follows default fetching policies for different types of associations
one-to-one : EAGER
one-to-many : LAZY
many-to-one : EAGER
many-to-many : LAZY

one-to-many : LAZY
Meaning : If you try to fetch details of one side(eg : Category) , will it fetch auto details of many side(blog posts) ?
NO (i.e select query will be fired only on Categorys table)
Why ? : for better performance

When will hibernate throw org.hibernate.LazyInitializationException ?
Any time you are trying to access un-fetched data from DB(represented by a proxy) , in a detached manner(outside the session scope)
Triggers : one-to-many
many-many
session's load method

un fetched data : i.e emp list in Category obj : represented by : proxy (substitution) : proxy object representing a collection
proxy => un fetched data from DB

Solutions
1. Change the fetching policy of hibernate for one-to-many to : EAGER
eg :
```
@OneToMany(mappedBy = "dept",cascade =
CascadeType.ALL,orphanRemoval=true,fetch=FetchType.EAGER)
 private List<BlogPost> emps=new ArrayList<>();
```

Is it recommneded soln : NO (since even if you just want to access one side details , hib will fire query on many side) --will lead to worst performance.
Use Case : when the size of many is small

OR
2.
```
 @OneToMany(mappedBy = "chosenCategory",cascade = CascadeType.ALL)
 private List<BlogPost> emps=new ArrayList<>();
```
Solution : Simply access the size of the collection within session scope : This soln will be applied in DAO layer
Dis Adv : Hibernate fires multiple queries to get the complete details

OR
3. Most recommended soln to avoid select n+1 issue :

How to fetch the complete details , in a single join query ?
Using "join fetch" keyword in JPQL
String jpql =

Hibernate will fire SINGLE INNER JOIN query to fetch category n blog post details (not resulting in LazyInitializationException)

Another trigger for lazy init exception
: Session's API
load.

2 triggers
1.Any-To-Many(one-many n many-many) --hibernate rets collection of proxies
2.Session's load method ---hibernate rets single proxy.

8. get vs load

9. Proceed to one-to-one
one-one uni directional association between User n Address
User 1-->1 Address
How will you configure ?
Will you use cascading ?

9.1 Assign user address
i/p -- adress details + user id

9.2 Lab work
Get user details along with it's address
i/p - user email
o/p -- user +adress detail

9.3 List all user names from a specified city
i/p city name
o/p -- list of users(names)

10. Many -Many